

Project Report

Bank Management System

Designed for
Object Oriented Programming Lab
Section C

Submitted in partial fulfillment of the requirements for the award of degree of
Bachelor of Science
in
Computer Science

Submitted by

Saad Mughal

F2023-009

Ramsha Naveed

F2023-026

Sara Haider

F2023-744

Khadijah Zahoor

F2023-956

Submitted to
Sir Ubaidullah Azeem



School of Computer & IT

Beaconhouse National University

Main Campus 13 KM Off Thokar Niaz Beg -
Raiwind Road, Tarogil, Lahore, Pakistan

1. How did you approach the problem?

To address the problem, We developed a "Banking Management System" with key classes:

- USER
- USER CLIENT
- BANKING EMPLOYEE
- TRANSACTION
- CARD
- COMPANY CLIENT
- ACCOUNT TYPE (with specific subtypes: GOLD, BUSINESS, and BRONZE)

Step-by-Step Approach:

- **Understanding the Requirements:**

At the start of the project, we carefully analyzed the problem requirements. This involved identifying the key features the system needed, such as account creation, a login system, and the ability to handle multiple user roles (e.g., clients, employees, and companies).

- **UML Diagram Creation:**

We designed a "UML Diagram" to map out the relationships between the various classes. The diagram served as a blueprint, allowing us to understand how each class interacted with others. We began with a dummy class diagram and refined it based on the system's specific requirements.

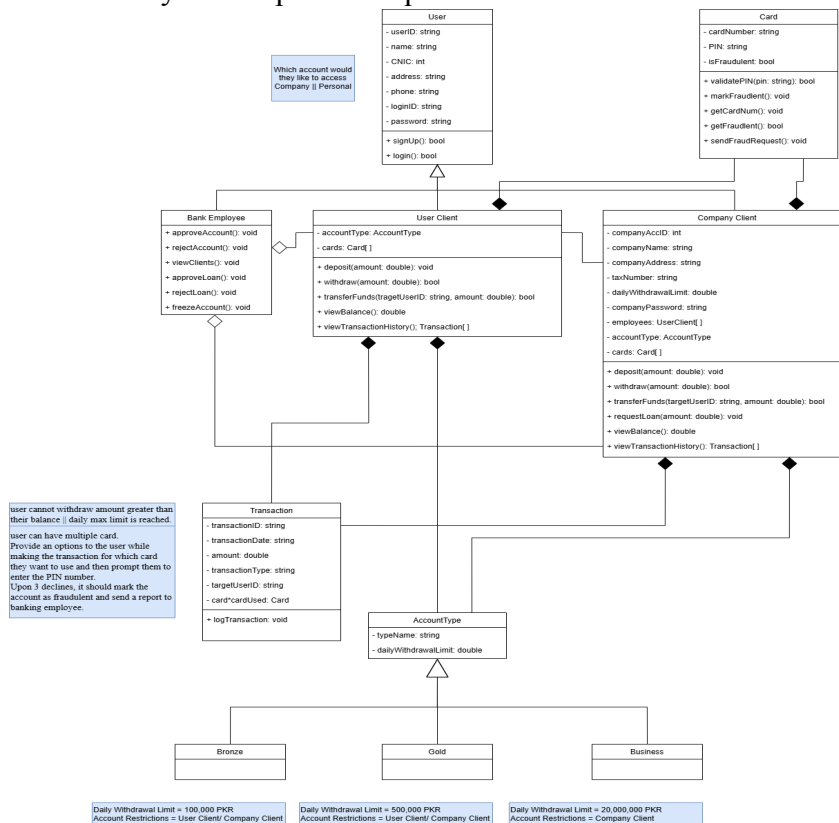


FIG. UML DIAGRAM FOR BANKING MANAGEMENT SYSTEM

- **Identifying Key Functionalities:**

We pinpointed the core functionalities required for the system, such as:

- ❖ Account Creation: Enabling users to create accounts with different types.
- ❖ Login System: Implementing login functionality for three distinct roles (user client, banking employee, and company client).
- ❖ Transaction Handling: Managing transactions and linking them to specific accounts.

- **Data Management:**

To store and manage data, we used text files for simplicity and modularity. These files included:

- ❖ `transaction.txt` for storing transaction records.
- ❖ `companies.txt` for company-related information.
- ❖ `cards.txt` for card details.
- ❖ `users.txt` for user account data.
- ❖ `company\_employees.txt` for information about employees linked to company clients.

- **Implementation:**

With the requirements clear and the class structure defined, we began coding the system. We ensured that each class encapsulated its specific responsibilities and interacted with other classes as defined in the class diagram. For instance:

- ❖ The TRANSACTION class handled all transaction-related operations.
- ❖ The CARD class managed card generation and validation.
- ❖ The ACCOUNT TYPE subclasses (GOLD, BUSINESS, BRONZE) defined specific account-related behaviors.

- **Testing and Refinement:**

After the initial implementation, we tested the system to ensure all functionalities worked as expected. This included verifying the login system, data storage in the respective files, and the seamless interaction between the classes.

2. What problems did you face while connecting to an external database?

- Authentication and Authorization Failures: Incorrect credentials or insufficient user permissions can block access to the database.
- Improper Connection Strings: Errors in the connection string (e.g., incorrect host, port, or database name) can prevent successful database connections.
- Network Connectivity Issues: Firewall restrictions, unstable internet connections, or blocked ports can prevent the system from connecting to the external database.
- SQL Query Errors: Syntax errors, logical mistakes, or incorrect data types in queries can result in failures when interacting with the database.
- Concurrency Problems: Simultaneous access to the database without proper transaction management or locking mechanisms can lead to data corruption or inconsistent results.

- Data Integrity Issues: Inserting or updating incorrect or invalid data (e.g., mismatched formats or missing fields) can break the database's integrity.
- Timeout Problems: Slow queries or network issues can lead to connection timeouts, interrupting the database operation.
- Database Schema Design Errors: Incorrect relationships or missing tables in the database schema can result in failures or data inconsistencies when querying the database.

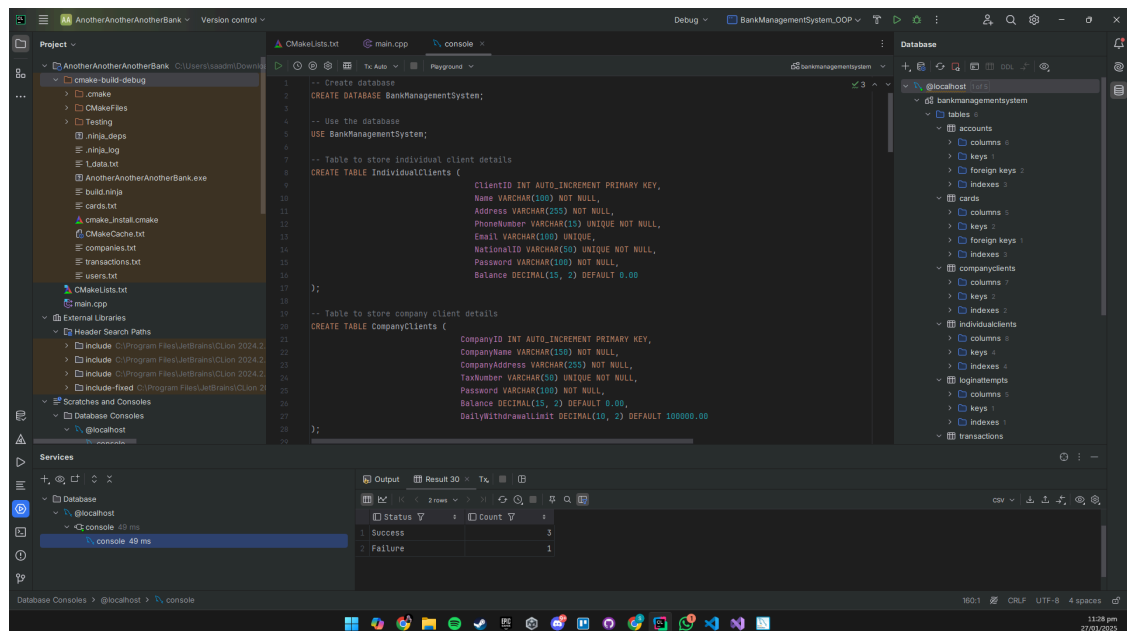
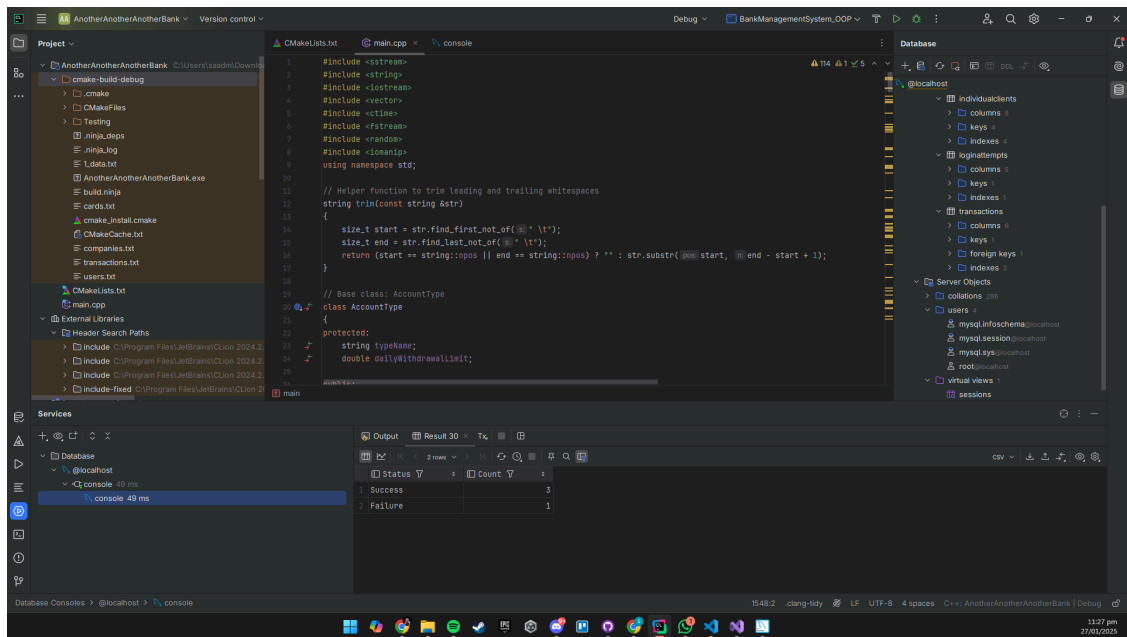


Fig 2. & Fig 3. SQL Integration

3. What was your motivation behind your particular management system (also applies to students working on banking management systems)?

The development of the Banking Management System is driven by the goal of applying software development concepts in a practical and meaningful way. This project allows us to deepen our understanding of essential principles such as modularity, data handling, and error management, while also enhancing our skills in C++ programming. The system is designed to perform key banking operations like account registration, secure login, and transactions in a simple and efficient manner.

Since banking systems are an integral part of daily life, working on this project provides valuable insights into how these systems handle secure transactions, manage customer data, and solve real-world problems. It also challenges us to address critical aspects like data validation, error handling, and ensuring smooth performance for multiple users.

In addition, this project lays the groundwork for creating more advanced systems in the future, such as online banking platforms or mobile banking applications. It also opens opportunities to integrate modern technologies like artificial intelligence for fraud detection and blockchain for enhanced security. Overall, this project demonstrates our ability to design and implement a functional system while showcasing our understanding of object-oriented programming, database management, and problem-solving—key skills that are crucial in software development.

4. How would you scale your system in the future?

While the current system is designed to meet the basic needs of managing user and company accounts, there are several ways the system can be scaled in the future to accommodate growth and enhance its functionality.

- **Web and Mobile Application**

We would develop a web or mobile application to allow users to access their accounts and perform transactions from anywhere. The app would offer a more intuitive and user-friendly interface, supporting features such as:

- ❖ Real-time account balance updates.
- ❖ Push notifications for account activities.
- ❖ Simplified access to transaction histories.

- **Security Enhancements**

The security of the system can be strengthened by integrating multi-factor authentication (MFA) for both users and banking employees. Additionally, encrypting sensitive account information, such as PINs and transaction details, would further protect user data.

- **Automated Loan System**

In the future, we would automate the loan approval process for company clients, incorporating predefined criteria like company size, financial history, and transaction volumes. This would streamline loan management and reduce administrative overhead.

- **Handling Multiple Branches**

To scale the system for larger banks with multiple branches, we would design a distributed architecture. This would allow each branch to manage its users and transactions while syncing with a central database to ensure data consistency.

- **Advanced Transaction Features**

Future updates could include the ability to process complex transactions such as international transfers, recurring payments, and detailed spending reports for individual and company clients.

SQL Integration

The screenshot shows the MySQL Workbench interface with the 'bankmanagementsystem' database selected in the left-hand 'SCHEMAS' pane. The 'Query' tab is active, displaying a series of SQL commands for database initialization. The 'Output' pane at the bottom shows the execution results of these commands, including creating users, granting privileges, creating a database, and showing databases.

```
1 CREATE USER 'root2'@'localhost' IDENTIFIED BY 'root2';
2 GRANT ALL PRIVILEGES ON *.* TO 'root2'@'localhost' WITH GRANT OPTION;
3 CREATE USER 'root2'@'192.168.1.10' IDENTIFIED BY 'root2';
4 GRANT ALL PRIVILEGES ON *.* TO 'root2'@'192.168.1.10' WITH GRANT OPTION;
5 FLUSH PRIVILEGES;
6
7 CREATE DATABASE quickstartdb;
8
9 USE quickstartdb;
10
11 SHOW DATABASES;
```

#	Time	Action	Message	Duration / Fetch
15	18:37:13	GRANT ALL PRIVILEGES ON *.* TO 'root2'@'localhost' WITH GRANT OPTION	0 row(s) affected	0.000 sec
16	18:37:16	CREATE USER 'root2'@'192.168.1.10' IDENTIFIED BY 'root2'	0 row(s) affected	0.015 sec
17	18:37:18	GRANT ALL PRIVILEGES ON *.* TO 'root2'@'192.168.1.10' WITH GRANT OPTION	0 row(s) affected	0.016 sec
18	19:05:33	FLUSH PRIVILEGES	0 row(s) affected	0.000 sec
19	19:06:46	CREATE DATABASE quickstartdb	1 row(s) affected	0.015 sec
20	19:17:58	USE quickstartdb	0 row(s) affected	0.000 sec
21	19:18:12	SHOW DATABASES	6 row(s) returned	0.000 sec / 0.000 sec

The screenshot shows the MySQL Workbench interface with the 'bank_db' database selected. The 'Query' tab is active, displaying three SQL queries. The 'Output' pane shows the results of these queries. A terminal window is also open, displaying a command-line interface for the 'bankmanagementsystem' database, showing a list of transactions and a banking menu.

```
1 SELECT * FROM quickstartdb.users;
2 SELECT * FROM quickstartdb.inventory;
3 SELECT * FROM quickstartdb.transactions;
```

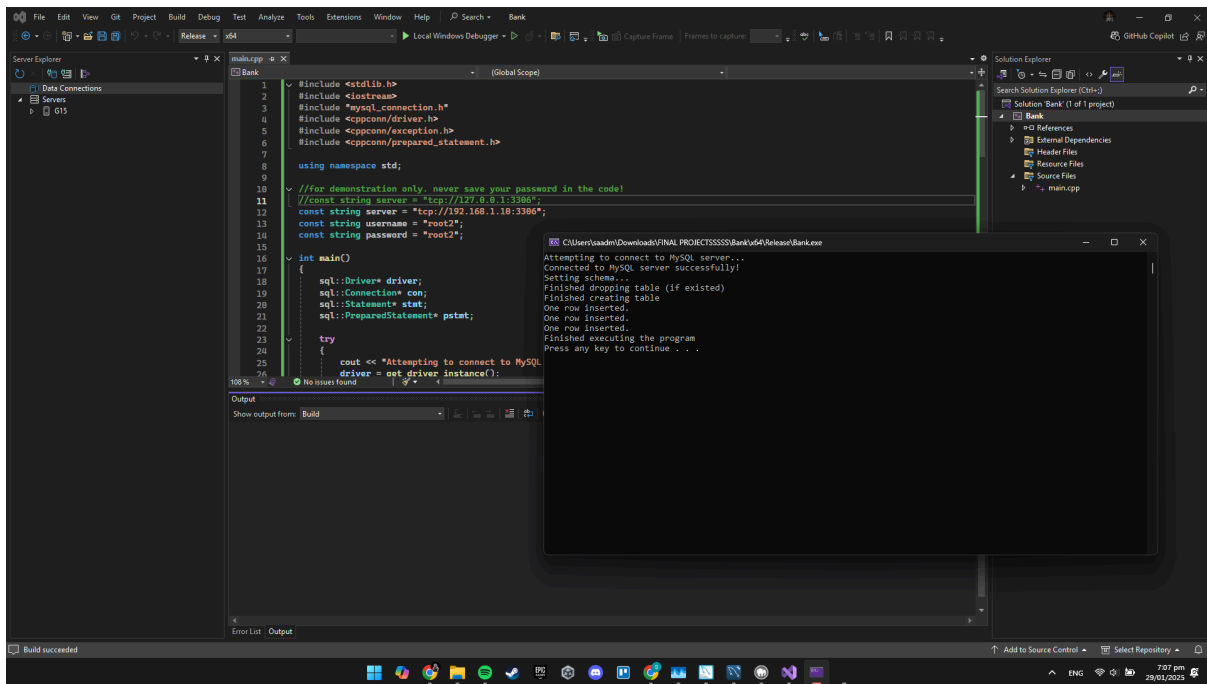
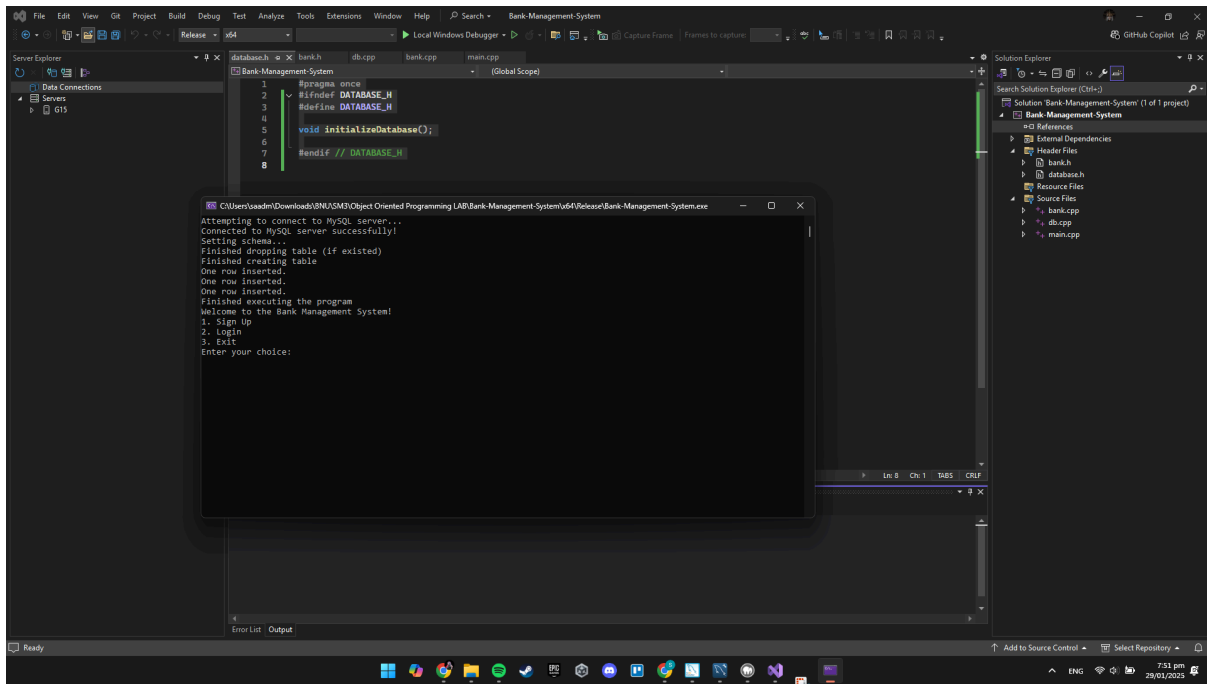
transactionID	userID	amount	transactionType	transactionDate	targetUserID
1	4	50000	Deposit	2025-01-29 22:03:43	
2	4	4500	Transfer	2025-01-29 22:03:52	1

```
1. Deposit Money
2. Withdraw Money
3. Transfer Funds
4. View Balance
5. View Transaction History
6. Logout
Enter your choice: 1
Enter amount to deposit: 50000
Deposit successful!

Banking Menu:
1. Deposit Money
2. Withdraw Money
3. Transfer Funds
4. View Balance
5. View Transaction History
6. Logout
Enter your choice: 3
Enter target user ID: 1
Enter amount to transfer: 4500
Transfer successful!

Banking Menu:
1. Deposit Money
2. Withdraw Money
3. Transfer Funds
4. View Balance
5. View Transaction History
6. Logout
Enter your choice:
```

#	Time	Action	Message	Duration / Fetch
23	21:58:12	SELECT * FROM quickstartdb.transactions LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
24	22:02:28	SELECT * FROM quickstartdb.users LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
25	22:02:28	SELECT * FROM quickstartdb.inventory LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
26	22:02:28	SELECT * FROM quickstartdb.transactions LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
27	22:03:56	SELECT * FROM quickstartdb.users LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
28	22:03:56	SELECT * FROM quickstartdb.inventory LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
29	22:03:56	SELECT * FROM quickstartdb.transactions LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec



MySQL Workbench

File Edit View Query Database Server Tools Scripting Help

Navigator

Filter objects

bank_db

Tables

Views

Stored Procedures

Functions

bankmanagementsystem

quickstartdb

Inventory

Transactions

Users

Views

Stored Procedures

Functions

saadm

Administration Schemas

Information

No object selected

Query 1 users - Table quickstartdb.users inventory users inventory users

1 SELECT * FROM quickstartdb.users;

2 SELECT * FROM quickstartdb.inventory;

3 SELECT * FROM quickstartdb.transactions;

Result Grid

userID	name	CNIC	address	phone	username	password
1	Saad	3520263149015	email@gmail.com	3218807891	Saad	password
2	Sara Hader	3520214512865	email@gmail.com	3218466154	Sara	smbu
3	Khadjah Zahoor	3520278951546	email@gmail.com	3218465476	Khadjah	codo
4	Ramsha Naveed	3520246851358	email@gmail.com	3214321187	Ramsha	catman

users 14 x inventory 15 transactions 16

Output

Action Output

#	Time	Action	Message	Duration / Fetch
20	21:55:36	SELECT * FROM quickstartdb.transactions LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
21	21:58:12	SELECT * FROM quickstartdb.users LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
22	21:58:12	SELECT * FROM quickstartdb.inventory LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
23	21:58:12	SELECT * FROM quickstartdb.transactions LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
24	22:02:28	SELECT * FROM quickstartdb.users LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
25	22:02:28	SELECT * FROM quickstartdb.inventory LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
26	22:02:28	SELECT * FROM quickstartdb.transactions LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec

Object Info session

C:\Users\saadm\Downloads\BNA19M3\Object Oriented Programming LAB\Bank-Management-System\Bank-Management-System.exe

Account successfully created!

Welcome to the Bank Management System!

1. Sign Up

2. Login

3. Exit

Enter your choice: 1

Enter Name: Khadijah Zahoor

Enter CNIC (13-digit): 3528278951546

Enter Address: email@gmail.com

Enter Phone: 3218465476

Enter Username: Khadijah

Enter Password: codo

Account successfully created!

Welcome to the Bank Management System!

1. Sign Up

2. Login

3. Exit

Enter your choice: 1

Enter Name: Ramsha Naveed

Enter CNIC (13-digit): 3528246851358

Enter Address: email@gmail.com

Enter Phone: 3214321187

Enter Username: Ramsha

Enter Password: catman

Account successfully created!

Welcome to the Bank Management System!

1. Sign Up

2. Login

3. Exit

Enter your choice: